arXiv:2608.04962v2 [cs.LG] 14 Oct 2026

# SPARK: Learning from Verification in Speculative GRPO Rollouts

**Nhat Minh Pham**[1,2,*] **Duy Tung Doan**[2,*] **Quang Huy Phung**[2] **Thi Duyen Ngo**[1]
**Vinh Van Nguyen**[1] **Khac-Hoai Nam Bui**[2]

[1]VNU University of Engineering and Technology, Vietnam National University, Hanoi, Vietnam
[2]Viettel AI, Viettel Group, Hanoi, Vietnam
{minhpn.266, duytunghanam}@gmail.com
[*]Equal contribution

## Abstract

Group Relative Policy Optimization (GRPO) has become a standard approach for improving the reasoning abilities of large language models, but its training is costly because each step requires generating many long reasoning responses. Recent methods use speculative decoding to accelerate the rollouts, where a draft model proposes tokens that the target model verifies. However, we find that the draft model repeatedly misses the same tokens within a response and across responses in the same GRPO group, and the verification feedback on these misses is not used to update the draft during the rollout. Motivated by this analysis, we introduce SPARK, a lightweight self-adaptive mechanism designed to enhance draft models used in speculative GRPO training. SPARK leverages target verification feedback to refine draft predictions during generation while preserving their existing training procedures. Averaged across 21 model–dataset settings, SPARK improves generation speedup by 32.7% and 11.4%, and end-to-end GRPO training speedup by 32.2% and 8.5%, over EAGLE3-style and DFlash-style baselines, respectively.

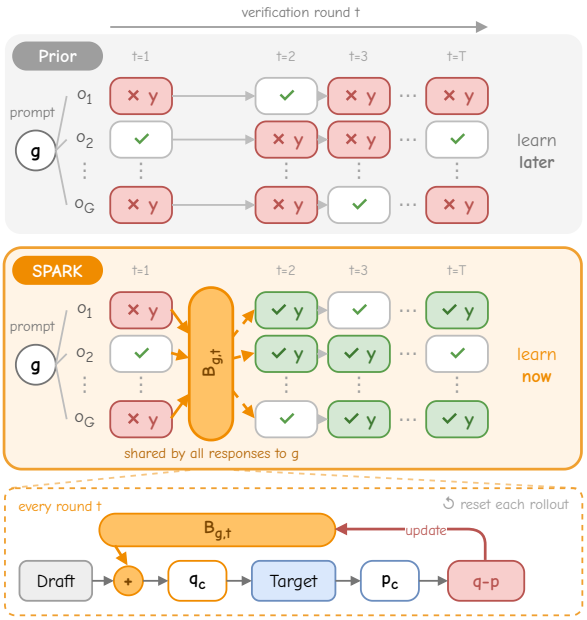



Figure 1: **Learning from verification during the rollout.** In GRPO, we find that the draft model often misses the same token $y$ again within a response and across responses to the same prompt $g$. Prior methods learn from these misses only after the rollout, so they keep repeating. SPARK turns each verification round into a small correction $B_{g,t}$ shared by all responses to $g$, so a miss in one round helps every response in the next.

## 1 Introduction

Group Relative Policy Optimization (GRPO) has become a standard approach for improving the reasoning abilities of large language models (Shao et al., 2024), but rollout generation dominates its training time: each step samples a group of long responses for every prompt. Speculative decoding reduces this cost by letting a lightweight draft model propose tokens that the target model verifies in parallel, preserving its sampling distribution.

Speculative RL systems obtain draft proposals in two ways. Non-parametric methods reuse generated text, either earlier trajectories of the same prompt (Liu et al., 2026) or a suffix tree over recent rollouts (Shao et al., 2025), so they help only when text repeats verbatim. Learned drafters generalize beyond exact repeats and are trained online to track the changing policy (Zhang et al., 2026; Chen et al., 2026b; Hu et al., 2026; Iso et al., 2026; Wang et al., 2026). However, these learned drafters are primarily improved through parameter updates rather than adjustments made during the ongoing rollout.

Within a rollout, they ignore the feedback that verification produces every round (Figure 1, top): beyond deciding which proposals to accept, verification reveals where the draft distribution departs from the target distribution. Whether this information helps later predictions is not obvious: some tokens may simply be difficult for the draft regardless of the rollout. We therefore study draft misses

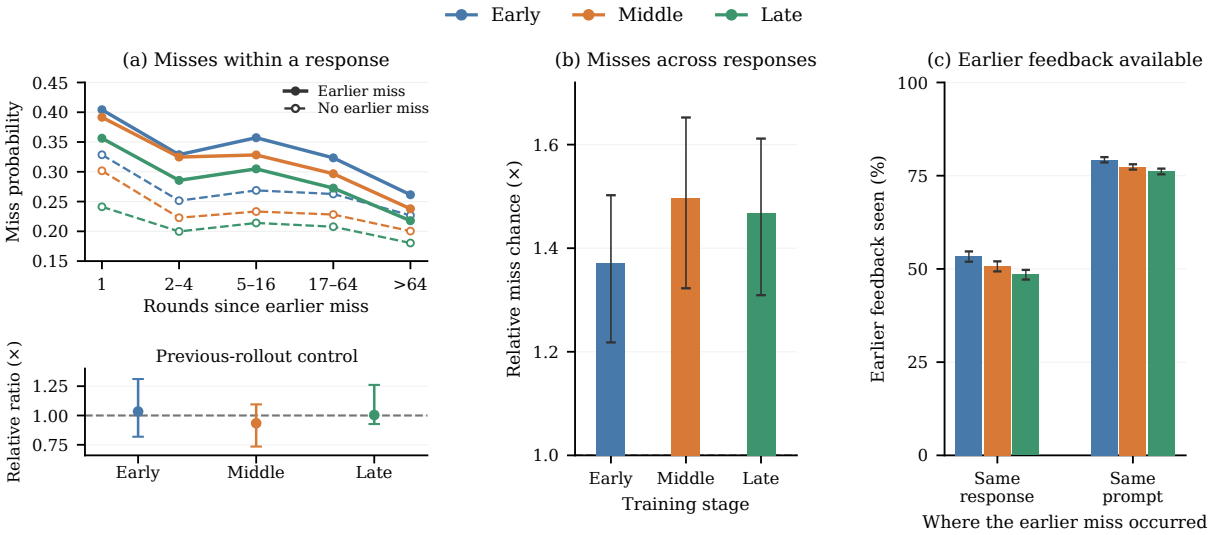



Figure 2: **Recurring draft misses during GRPO rollouts.** (a) The upper plot compares miss probability when a token was missed earlier in the same response and when it was not. The lower plot measures whether an earlier miss in an independent previous rollout predicts another miss in the current rollout; values near $1\times$ indicate little difference. (b) Relative miss probability after an earlier miss on the same token in another response generated from the same prompt. (c) Percentage of misses preceded by an earlier miss on the same token within the response or within the prompt group.

to determine whether earlier verification feedback predicts later errors. As shown in Figure 2 and examined in Section 2, misses recur within a response, while misses from an independent previous rollout show little association. We also find that misses are related across responses generated from the same prompt, and that many later misses involve tokens already missed earlier during generation.

We introduce SPARK (Figure 1, middle and bottom), which allows draft predictions to adapt using verification feedback during the rollout. SPARK adds a low-rank correction to the draft logits, shared within each prompt group and updated after each verification round using target probabilities computed by the verifier. The correction is reset at the start of each rollout, while a projection from draft features is retained and trained with the draft model across rollouts. SPARK requires no additional target forward passes or draft backbone parameter updates during generation, and preserves the verification and draft training procedures.

We evaluate SPARK across models, datasets, and speculative drafting methods. Across 21 model–dataset settings, SPARK achieves average generation speedups of $2.43\times$ and $2.30\times$ and average end-to-end GRPO training speedups of $2.31\times$ and $2.11\times$ with EAGLE3- and DFlash-style drafting, respectively. Relative to their corresponding baselines, SPARK improves generation speedup by 32.7% and 11.4% and end-to-end speedup by 32.2% and 8.5%, respectively. SPARK retains the exact target-distribution sampling guarantee.

Our contributions are:

- **Analysis.** We analyze draft misses during GRPO rollouts and show that earlier verification outcomes are associated with later errors, while an independent previous-rollout control shows little comparable association. This reveals opportunities to use verification feedback before generation ends.
- **Method.** We introduce SPARK, a lightweight adaptation mechanism for draft models used in speculative GRPO training. SPARK leverages target verification feedback to refine draft predictions during generation, without additional target model forward passes or draft backbone parameter updates, while preserving existing online draft training procedures.
- **Evidence.** We evaluate draft acceptance, generation speedup, and end-to-end GRPO training efficiency across different models, datasets, and drafting methods. Ablations examine feedback timing, sharing scope, feedback selection, and projection learning. SPARK improves draft acceptance and training efficiency (Table 1), with ablations supporting its design choices (Table 2).

## 2 Analyzing Verification Feedback During GRPO Rollouts

Each speculative verification round provides target probabilities and reveals where the draft predictions disagree with the target. However, this feedback is not typically used to improve subsequent draft proposals during the same rollout. We therefore examine draft misses to understand whether verification provides information that could be useful before generation ends. A miss occurs when the draft proposals fail to include the token selected by the target. We investigate whether the same mistakes reappear within a response, across responses, and whether earlier feedback is available before later misses occur. Our experiments use Qwen2.5-3B-Instruct trained with GRPO on SimpleLR (Zeng et al., 2025)[1], with a single-layer EAGLE3-style drafter. We evaluate draft misses at early, middle, and late training stages using a consistent rollout sampling setup. Figure 2 summarizes the results.

### 2.1 Misses Recur Within a Response

We first ask whether the draft is more likely to miss a token again after missing it earlier in the same response. We compare the probability of a miss when the token was previously missed with the probability when it was not, at different distances between verification rounds.

Figure 2(a, top) shows that tokens missed earlier have a consistently higher chance of being missed again, even after more than 64 rounds. One possible explanation is that some tokens are simply difficult for the draft, regardless of the rollout. To examine this, we also use misses recorded in an independent previous rollout. In Figure 2(a, bottom), we divide the miss probability for tokens missed in the previous rollout by the probability for comparable tokens without such an earlier miss. A ratio of $1\times$ means that both groups have the same miss probability. The measured ratios are $1.03\times$, $0.93\times$, and $1.00\times$ at the early, middle, and late stages. Unlike the clear recurrence within a response, misses from a previous rollout do not consistently predict more misses in the current one. This suggests that feedback from the ongoing rollout is especially relevant to later draft predictions.

[1] https://huggingface.co/datasets/Feierabend/simplelr_abel_level3to5

### 2.2 Misses Are Shared Within a Prompt Group

We next examine whether a miss in one response is related to misses in other responses generated from the same prompt. We identify repeated errors by matching a single target token across responses. As above, we measure the ratio between the probability of a miss when the same token was missed earlier in another response and the probability when it was not.

Figure 2(b) shows ratios of $1.37\times$, $1.50\times$, and $1.47\times$ at the early, middle, and late stages. In other words, if a token has already been missed in one response, it becomes substantially more likely to be missed again in another response to the same prompt. This result motivates sharing verification feedback among responses generated from that prompt.

### 2.3 Earlier Feedback Is Available During Generation

Finally, for earlier feedback to help later draft predictions, it must be available before the next miss occurs. For each miss, we check whether the same token was already missed earlier in the current response or in another response generated from the same prompt.

Figure 2(c) shows that, at the late training stage, $48.5\%$ of misses had an earlier occurrence within the same response, and $76.2\%$ had an earlier occurrence within the same prompt. Similar patterns appear at the early and middle stages. Together with the results above, these findings show that verification often provides feedback before similar draft errors occur again, motivating the use of that information during generation rather than only in subsequent draft training.

## 3 SPARK: Learning from Verification Feedback During the Rollout

Section 2 shows that draft misses recur within a response and across responses to the same prompt, but not across rollouts. The useful feedback is therefore produced during the rollout and must be used before it ends. SPARK (Figure 3) adds a low-rank correction to the draft logits that is updated after every verification round and reset at the end of the rollout, on top of a projection learned across rollouts. Formal statements and proofs of its properties are given in Appendix B.

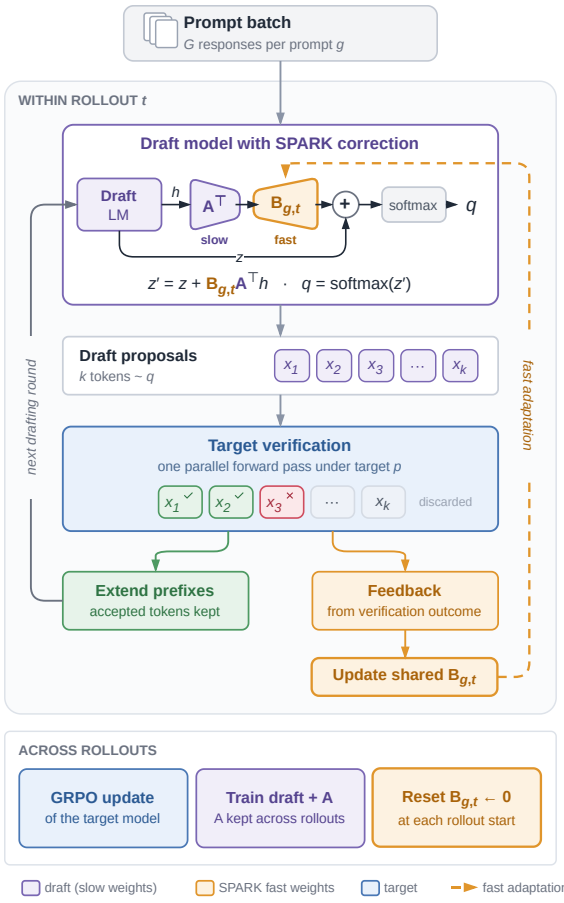



Figure 3: **Overview of SPARK.** During a rollout, the draft logits are corrected by $B_{g,t}A^\top h_c$, where $A$ is a projection learned across rollouts and $B_{g,t}$ is a correction matrix shared by the responses to prompt $g$. After each verification round, target distributions already computed by the verifier update $B_{g,t}$. At the next rollout, $B_{g,t}$ is reset to zero, while $A$ is retained and trained with the draft model.

### 3.1 Preliminaries and Notation

A *rollout* is the generation phase of one GRPO step, in which a group of responses is sampled for each prompt. Generation proceeds in verification *rounds*: in round $t$, the drafter proposes tokens for each active response, and the target scores them in one forward pass. A *context* $c$ is a position where the drafter predicts a next token, and $g(c)$ is the prompt of its response. At $c$, $h_c \in \mathbb{R}^d$ is the feature entering the draft output head, $z_c \in \mathbb{R}^V$ are the draft logits, and $p_c$ is the target distribution used by verification.

### 3.2 A Rollout-Local Logit Correction

Motivated by the shared miss patterns across responses to the same prompt (§2.2), SPARK maintains a prompt-specific correction matrix $B_{g,t} \in \mathbb{R}^{V\times r}$ and a projection $A \in \mathbb{R}^{d\times r}$ shared across prompts, where $r \ll d$. At round $t$, the draft logits are adjusted as

$$\widetilde{z}_c = z_c + B_{g(c),t}\, u_c, \qquad u_c = A^\top h_c, \tag{1}$$

and proposals are sampled from $q_c = \mathrm{softmax}(\widetilde{z}_c)$; proposal construction and verification otherwise remain unchanged. The projected features $u_c$ make the correction context-dependent even though $B_{g,t}$ is shared by the prompt (Proposition B.5), while the low-rank parameterization limits computation and memory. Because misses from previous rollouts offer limited predictive signal (§2.1), we set $B_{g,0} = 0$ at the start of each rollout and retain only $A$ across rollouts. We evaluate $r \in \{4, 8, 16, 32, 64\}$ in Appendix C.

### 3.3 Updating the Correction Every Round

Target verification provides $p_c$, which is ordinarily used only to accept or reject draft proposals. SPARK additionally uses it to align $q_c$ with $p_c$. Since a round terminates when the proposals fail to cover the target's token, we minimize the forward KL divergence $\mathcal{L}_c = D_{\mathrm{KL}}(p_c \,\|\, q_c)$, which, unlike the reverse KL, strongly penalizes target mass left uncovered by the draft and bounds the miss probability (Proposition B.2). Its gradient with respect to the corrected logits is (Lemma B.1)

$$\frac{\partial \mathcal{L}_c}{\partial \widetilde{z}_c(v)} = q_c(v) - p_c(v). \tag{2}$$

Let $C_{g,t}$ denote the draft positions selected to update $B_{g,t}$ after round $t$ for prompt $g$. We include positions along the verified path and rejected proposals directly adjacent to it. For tree-based drafting, these include rejected branches neighboring the verified path; for block-based drafting, only the first rejected token is retained. Positions beyond a rejection or without a corresponding draft prediction are excluded. To obtain sparse feedback, we retain the coordinates in

$$S_c = \mathrm{TopK}(p_c, k) \cup \mathrm{TopK}(q_c, k), \tag{3}$$

which covers tokens the draft may overestimate or underestimate, yielding

$$\delta_c = (q_c - p_c) \odot \mathbf{1}_{S_c}. \tag{4}$$

This sparse feedback still gives a descent direction, with an error bounded by the probability mass outside $S_c$ (Proposition B.4). We assess $k$ in Appendix C.

Since the corrected logits are linear in $B_{g,t}$, the per-context gradient is $\delta_c u_c^\top$ (Lemma B.3). SPARK applies one gradient step to the weighted average loss over $C_{g,t}$:

$$B_{g,t+1} = B_{g,t} - \eta \sum_{c \in C_{g,t}} \frac{w_c}{W_{g,t}} \delta_c u_c^\top, \qquad W_{g,t} = \sum_{c \in C_{g,t}} w_c, \tag{5}$$

where $w_c \geq 0$ is the feedback weight and $\eta$ is the step size.[2] This explicit update requires no backward pass or additional target forward pass, and the resulting $B_{g,t+1}$ is used from the next round onward. Unrolled from $B_{g,0} = 0$, the correction at a new context is a sum of earlier feedback weighted by the similarity of projected features, and it changes only the logits of tokens that verification has already exposed for that prompt (Proposition B.5).

### 3.4 Learning the Projection Across Rollouts

Whereas $B_{g,t}$ captures rollout-specific errors, $A$ learns a feature subspace for logit correction that is shared across prompts and rollouts. We initialize $A$ from the target model's output projection $W \in \mathbb{R}^{V \times d}$: we take $r$ evenly spaced rows of $W$ and orthonormalize them by QR decomposition, so that the columns of $A$ form an orthonormal basis of a subspace spanned by token embeddings. $A$ is then updated throughout training. Treating $B_{g(c),t}$ as fixed, the gradient contribution of context $c$ is (Lemma B.3)

$$G_c^A = h_c \big(B_{g(c),t}^\top \delta_c\big)^\top. \tag{6}$$

Because $B_{g,0} = 0$, $A$ receives gradient only after within-rollout adaptation has begun, so it learns from the corrections actually required during generation; to first order, each step increases the similarity between contexts whose feedback agrees (Remark B.6). These gradients are weighted by $w_c$, accumulated during generation, and applied at the existing draft optimizer step. Both $A$ and the draft backbone remain fixed during generation, and GRPO is unchanged. SPARK therefore operates on two timescales: $B_{g,t}$ adapts within a rollout, while $A$ is learned with the draft model across rollouts.

SPARK modifies only the proposal distribution. Exact speculative decoding preserves the target distribution for any proposal distribution fixed before verification, provided that the acceptance rule uses the corresponding $q_c$. Since $B_{g,t}$ changes only between verification rounds, the proposals of each round are fixed before they are verified, and the guarantee holds (Theorem B.8); hence the distribution of the GRPO update is also unchanged (Corollary B.9). Individual trajectories may differ from the baseline, but they follow the same target distribution. The correction costs $O(dr + Vr)$ per context, compared with $O(Vd)$ for the draft output head, and requires one $V \times r$ matrix per prompt and one shared $d \times r$ projection (Remark B.11).

[2] If $W_{g,t} = 0$, $B_{g,t}$ remains unchanged.

## 4 Experiments and Results

We evaluate SPARK by addressing three research questions:

RQ1: Does SPARK improve generation efficiency and end-to-end GRPO training speed across different speculative decoding methods?

RQ2: Does SPARK reduce the recurring draft misses identified in Section 2, and how does its benefit change during a rollout?

RQ3: How do feedback timing, sharing scope, feedback selection, and the learned projection affect performance?

### 4.1 Experimental Setup

**Models and datasets.** We evaluate seven models: Qwen2.5-1.5B-Instruct, Qwen2.5-3B-Instruct, Qwen2.5-7B-Instruct, Qwen2.5-14B-Instruct, Qwen3-1.7B, Qwen3-4B, and Llama3.1-8B-Instruct. Each model is trained on GSM8K (Cobbe et al., 2021), SimpleLR (Zeng et al., 2025), and DAPO-Math-17K (Yu et al., 2025), yielding 21 model–dataset settings. Complete training configurations are provided in Appendix A.

**Draft pretraining.** We pretrain each draft model for one epoch on ShareGPT[3] before GRPO training. Within each baseline–SPARK pair, both runs start from the same target and pretrained draft checkpoints.

**Drafting methods.** We use two drafting setups, referred to as *EAGLE3-style* and *DFlash-style*. EAGLE3-style uses the single-layer EAGLE3-style drafter and online draft training procedure from FastGRPO (Zhang et al., 2026). DFlash-style uses the DFlash draft architecture (Chen et al., 2026a) and the online draft co-training implementation of

[3] https://huggingface.co/datasets/Aeala/ShareGPT_Vicuna_unfiltered

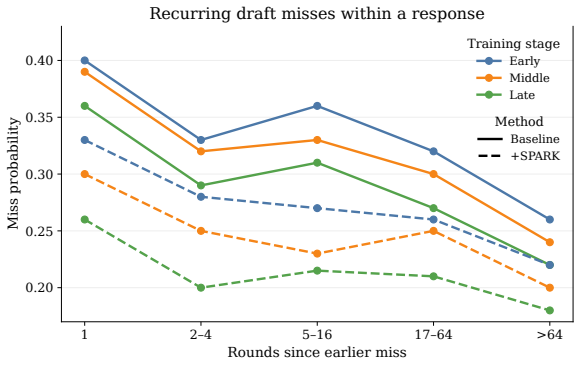



Figure 4: **Recurring draft misses within a response.** Miss probability across intervals of verification rounds after an earlier miss. Solid lines indicate the baseline drafter, dashed lines indicate SPARK, and colors denote early, middle, and late training stages.

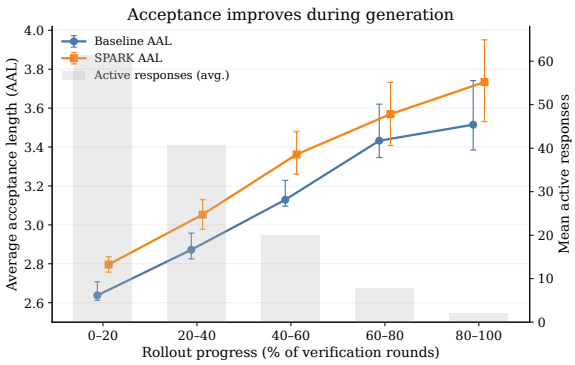



Figure 5: **Average acceptance length throughout a rollout.** Average acceptance length (AAL) with SPARK disabled and enabled over successive intervals of verification-round progress. Gray bars (right axis) show the average number of active responses across the two settings.

NVIDIA (Wang et al., 2026). For each setup, we evaluate the baseline drafter and its SPARK variant under the same model initialization and GRPO configuration. SPARK preserves the original draft training and target verification procedures.

**Metrics.** We report average acceptance length (AAL), generation speedup (Gen.), and end-to-end GRPO training speedup (E2E). Generation speedup compares rollout generation time against autoregressive GRPO for the same model and dataset, normalized by the amount of generated output. E2E is the ratio of autoregressive GRPO training time to the method's training time at the same number of target policy updates, including rollout generation, online draft training, and target optimization. Draft pretraining is excluded from these measurements and reported separately. Both speedup measures include SPARK's runtime overhead. Since the two drafting setups construct proposals differently, we interpret AAL primarily within each setup.

### 4.2 RQ1: Overall Effectiveness

Table 1 compares EAGLE3-style and DFlash-style drafting, each with and without SPARK, across model–dataset settings. We examine both generation speedup and end-to-end GRPO training speedup to evaluate the benefits of SPARK.

With EAGLE3-style drafting, SPARK achieves an average generation speedup of $2.43\times$ and an E2E speedup of $2.31\times$, improving upon the corresponding baseline by $32.7\%$ and $32.2\%$, respectively. With DFlash-style drafting, SPARK achieves average speedups of $2.30\times$ for generation and $2.11\times$ for E2E training, corresponding to improvements of $11.4\%$ and $8.5\%$ over the baseline. These results demonstrate that SPARK improves both rollout generation and overall training efficiency across different drafting architectures.

### 4.3 RQ2: Understanding the Improvements

**Recurring draft misses.** We investigate whether SPARK reduces the repeated draft misses observed in Section 2. Using the same frozen checkpoints, we compare how likely the baseline drafter and SPARK are to miss a token again after an earlier miss in the same response. Figure 4 compares these probabilities across training stages and distances between verification rounds. SPARK shows lower miss probabilities than the baseline throughout the measured intervals, indicating that verification feedback can help reduce repeated errors during generation.

**Improvements throughout generation.** We further investigate how SPARK improves draft acceptance as generation progresses. Using the same frozen target, drafter, and learned projection, we compare the baseline with SPARK to evaluate the benefit of incorporating verification feedback during each rollout. Figure 5 shows that SPARK consistently achieves higher average acceptance length (AAL) throughout generation. Averaging the AAL values plotted in the figure, SPARK improves AAL from 3.12 to 3.30, corresponding to a gain of $5.95\%$. This improvement demonstrates the benefit of adapting draft predictions using feedback from previous verification rounds, allowing SPARK to produce more accurate drafts without up-

Table 1: **Overall GRPO efficiency across seven models and three datasets.** For each draft method, we report generation speedup (Gen.) and end-to-end speedup (E2E), both relative to autoregressive GRPO, and average acceptance length (AAL), without (Orig.) and with SPARK (shaded). **Bold** marks the better result in each pair; ties at the reported precision are both bolded. The last row reports the mean over all 21 model–dataset settings, with ↑ indicating the relative improvement of the mean over Orig.

| Dataset | EAGLE3-style | | | | | | DFlash | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|
| | Gen. | | E2E | | AAL | | Gen. | | E2E | | AAL | |
| | Orig. | +SPARK | Orig. | +SPARK | Orig. | +SPARK | Orig. | +SPARK | Orig. | +SPARK | Orig. | +SPARK |
| *Qwen2.5-1.5B-Instruct* | | | | | | | | | | | | |
| GSM8K | 1.75 | **2.26** | 1.69 | **2.14** | 2.46 | **3.24** | 2.23 | **2.52** | 2.08 | **2.27** | 2.58 | **2.98** |
| SimpleLR | 1.66 | **2.14** | 1.61 | **2.03** | 2.37 | **3.12** | 2.13 | **2.26** | 1.98 | **2.06** | 2.58 | **2.81** |
| DAPO-Math-17K | 1.46 | **1.87** | 1.44 | **1.81** | 2.54 | **3.33** | 1.78 | **2.01** | 1.71 | **1.90** | 2.63 | **3.05** |
| *Qwen2.5-3B-Instruct* | | | | | | | | | | | | |
| GSM8K | 1.95 | **2.28** | 1.83 | **2.02** | 2.65 | **3.28** | 2.24 | **2.42** | 2.03 | **2.16** | 2.53 | **2.81** |
| SimpleLR | 1.60 | **2.68** | 1.54 | **2.53** | 2.66 | **2.77** | 1.71 | **1.84** | 1.66 | **1.75** | 2.50 | **2.78** |
| DAPO-Math-17K | 1.48 | **1.90** | 1.45 | **1.84** | 2.60 | **3.43** | 1.61 | **1.82** | 1.58 | **1.74** | 2.44 | **2.85** |
| *Qwen2.5-7B-Instruct* | | | | | | | | | | | | |
| GSM8K | 2.16 | **3.71** | 2.06 | **3.60** | 2.99 | **3.22** | 2.26 | **2.64** | 2.16 | **2.43** | 2.66 | **3.21** |
| SimpleLR | 1.54 | **2.53** | 1.36 | **2.52** | 3.09 | **3.31** | 1.51 | **1.72** | 1.49 | **1.66** | 2.82 | **3.29** |
| DAPO-Math-17K | 1.88 | **2.50** | 1.81 | **2.35** | 2.31 | **3.14** | 2.28 | **2.62** | 2.15 | **2.39** | 2.33 | **2.74** |
| *Qwen2.5-14B-Instruct* | | | | | | | | | | | | |
| GSM8K | 1.98 | **2.56** | 1.89 | **2.39** | 2.65 | **3.51** | 2.46 | **2.80** | 2.28 | **2.53** | 2.68 | **3.13** |
| SimpleLR | 3.48 | **3.62** | 3.16 | **3.33** | 3.05 | **3.85** | 3.53 | **3.93** | 3.14 | **3.37** | 2.96 | **3.39** |
| DAPO-Math-17K | 2.14 | **2.74** | 2.04 | **2.56** | 2.19 | **2.88** | 2.59 | **2.93** | 2.40 | **2.64** | 2.21 | **2.58** |
| *Qwen3-1.7B* | | | | | | | | | | | | |
| GSM8K | 1.41 | **1.92** | 1.39 | **1.84** | 2.53 | **3.54** | 1.60 | **1.80** | 1.55 | **1.70** | 2.42 | **2.81** |
| SimpleLR | 1.55 | **1.99** | 1.50 | **1.90** | 2.36 | **3.12** | 1.54 | **1.59** | 1.50 | **1.53** | 2.27 | **2.43** |
| DAPO-Math-17K | 1.76 | **2.23** | 1.69 | **2.10** | 2.47 | **3.20** | 1.94 | **2.17** | 1.83 | **2.00** | 2.38 | **2.72** |
| *Qwen3-4B* | | | | | | | | | | | | |
| GSM8K | 1.78 | **2.26** | 1.70 | **2.11** | 2.62 | **3.40** | 2.04 | **2.31** | 1.91 | **2.10** | 2.50 | **2.90** |
| SimpleLR | 1.63 | **2.09** | 1.58 | **1.97** | 2.59 | **3.40** | 1.60 | **1.61** | **1.55** | **1.55** | 2.46 | **2.57** |
| DAPO-Math-17K | 1.86 | **2.35** | 1.78 | **2.21** | 2.41 | **3.12** | 2.08 | **2.34** | 1.95 | **2.13** | 2.30 | **2.66** |
| *Llama3.1-8B-Instruct* | | | | | | | | | | | | |
| GSM8K | 1.75 | **2.22** | 1.68 | **2.09** | 2.17 | **2.83** | 2.01 | **2.27** | 1.89 | **2.06** | 2.19 | **2.55** |
| SimpleLR | 1.74 | **2.21** | 1.66 | **2.08** | 2.30 | **3.01** | 1.99 | **2.22** | 1.85 | **2.02** | 2.35 | **2.71** |
| DAPO-Math-17K | 1.98 | **2.50** | 1.86 | **2.32** | 1.89 | **2.46** | 2.21 | **2.48** | 2.05 | **2.23** | 1.90 | **2.20** |
| **Average** | 1.84 | **2.43**↑ 32.7% | 1.75 | **2.31**↑ 32.2% | 2.52 | **3.47**↑ 37.7% | 2.06 | **2.30**↑ 11.4% | 1.94 | **2.11**↑ 8.5% | 2.46 | **2.82**↑ 14.5% |

dating the underlying model weights. The gray bars illustrate how the number of active responses gradually decreases as generation progresses. Notably, SPARK maintains its advantage over the baseline even as fewer responses remain active. These results highlight the effectiveness of prompt-specific adaptation in improving draft acceptance throughout GRPO rollouts.

### 4.4 RQ3: Ablation Studies

We examine the key design choices of SPARK using Qwen2.5-3B-Instruct on GSM8K with EAGLE3-style drafting. Starting from the full method, we change one component at a time while keeping the remaining experimental settings fixed. Full SPARK shares a correction matrix $B_{g,t}$ among responses to the same prompt, updates it after every verification round, and resets it at the start of each rollout. It also uses feedback from both accepted and eligible rejected positions and learns the projection $A$ across rollouts. As shown in Table 2, full SPARK achieves the highest AAL (3.28) and end-to-end (E2E) training speedup (2.02×) among the evaluated configurations.

**Feedback update frequency.** Updating the correction after every verification round provides the best results. Reducing the update frequency to once every 5, 10, or 15 rounds decreases AAL from 3.28 to 3.11, 3.01, and 2.85, respectively. The corresponding E2E speedups also decline from 2.02× to 1.95×, 1.92×, and 1.84×. These results show that promptly incorporating verification feedback improves draft acceptance while also benefiting overall training efficiency.

**Correction sharing and reset.** We next examine how the correction is shared across responses. Assigning a separate correction to each response achieves an AAL of 3.24 and an E2E speedup of 1.96×, whereas sharing one correction across the entire rollout batch reduces them to 3.12 and 1.92×. The prompt-specific design performs best, allow-

Table 2: **SPARK ablation results** on Qwen2.5-3B-Instruct with GSM8K using EAGLE3-style drafting. Each ablation changes one component relative to full SPARK. E2E denotes end-to-end training speedup over autoregressive GRPO.

| Configuration | AAL | E2E (×) |
|---|---|---|
| **Full SPARK** | **3.28** | **2.02** |
| Update every 5 rounds | 3.11 | 1.95 |
| Update every 10 rounds | 3.01 | 1.92 |
| Update every 15 rounds | 2.85 | 1.84 |
| Per-response correction | 3.24 | 1.96 |
| Batch-wide correction | 3.12 | 1.92 |
| Carry correction across rollouts | 3.05 | 1.85 |
| Accepted feedback only | 3.02 | 1.94 |
| Frozen projection $A$ | 2.88 | 1.91 |

ing responses to the same prompt to benefit from shared feedback without mixing corrections across unrelated prompts. Resetting the correction between rollouts is also beneficial: carrying it across rollouts reduces AAL to 3.05 and E2E speedup to 1.85×. This supports adapting the correction to the current rollout rather than retaining it across different rollouts.

**Feedback selection and learned projection.** Using accepted positions alone reduces AAL from 3.28 to 3.02 and E2E speedup from 2.02× to 1.94×, highlighting the benefit of incorporating eligible rejected proposals. We also examine how the number of selected feedback tokens $k$ affects performance. Increasing $k$ from 8 to 16 improves both AAL and E2E speedup, while larger values provide little additional benefit. Although $k = 32$ achieves a slightly higher AAL (3.29), $k = 16$ delivers the highest E2E speedup (2.02×), offering a better balance between draft acceptance and training efficiency. Detailed results are provided in Appendix C. Finally, freezing the projection $A$ reduces AAL to 2.88 and E2E speedup to 1.91×, demonstrating the importance of learning the projection across rollouts.

## 5 Related Work

**Efficient RL training and speculative decoding.** Rollout generation dominates the cost of GRPO-style RL (Shao et al., 2024; Guo et al., 2025). Asynchronous and partial rollouts raise throughput at the cost of on-policyness (Fu et al., 2025; Kimi Team, 2025; Sheng et al., 2025), whereas speculative decoding preserves the target distribution (Leviathan et al., 2023; Chen et al., 2023). Drafters include auxiliary heads (Cai et al., 2024; Li et al., 2024, 2025), token trees (Miao et al., 2024), block drafters (Chen et al., 2026a), and retrieval (He et al., 2024; Oliaro et al., 2024). Learned drafters stay fixed during generation, and verification only accepts or rejects their proposals.

**Speculative decoding for RL.** RL drafters must track a changing policy. Non-parametric methods reuse earlier rollouts as drafts (Liu et al., 2026; Shao et al., 2025); DAS updates its suffix tree within a rollout but matches only verbatim repeats. Learned drafters are trained between rollouts or RL steps (Zhang et al., 2026; Hu et al., 2026; Chen et al., 2026b; Iso et al., 2026; Wang et al., 2026), including FastGRPO, whose concurrency-aware budget we adopt. None of them can correct errors that recur within the current rollout (Section 2).

**Online and test-time adaptation.** Online speculative decoding adapts drafter weights across queries (Liu et al., 2024), and DistillSpec aligns the drafter by distillation (Zhou et al., 2024). Dynamic evaluation, test-time training and semi-amortized inference adapt a model to each instance (Krause et al., 2018; Sun et al., 2020; Kim et al., 2018). SPARK brings this per-instance adaptation to speculative RL: to our knowledge, it is the first learned, distributional adaptation of the drafter within a rollout. After every verification round, it makes a closed-form low-rank update from target distributions the verifier has already computed, including at rejected proposals, and shares it within each GRPO group. It extends existing drafters without changing their training or the target distribution.

## 6 Conclusion

We introduced SPARK, a lightweight mechanism that uses verification feedback to adapt draft predictions during GRPO rollouts. Prompt-shared, low-rank corrections reduce recurring draft misses while preserving the target distribution and requiring no additional target forward passes. Across 21 model–dataset settings, SPARK improves end-to-end training speedup by 32.2% over EAGLE3-style drafting and 8.5% over DFlash-style drafting. These results show that feedback already available during verification can improve rollout efficiency beyond online draft training alone.

## Limitations

Our evaluation focuses on mathematical reasoning datasets (GSM8K, SimpleLR and DAPO-Math-17K). Other RL domains, such as code generation, agentic tasks or open-ended dialogue, may show different patterns of draft misses and remain to be evaluated. We apply SPARK to two drafting approaches, EAGLE3-style trees and DFlash blocks; other drafters, such as Medusa-style heads, separate small models or retrieval-based methods, could also be combined with SPARK but are not tested here. Our experiments use Qwen and Llama models of up to 14B parameters with GRPO. Larger models, other model families and other group-based RL algorithms are left to future work.

## References


Tianle Cai, Yuhong Li, Zhengyang Geng, Hongwu Peng, Jason D. Lee, Deming Chen, and Tri Dao. 2024. Medusa: Simple LLM inference acceleration framework with multiple decoding heads. In *Proceedings of the 41st International Conference on Machine Learning (ICML)*.

Charlie Chen, Sebastian Borgeaud, Geoffrey Irving, Jean-Baptiste Lespiau, Laurent Sifre, and John Jumper. 2023. Accelerating large language model decoding with speculative sampling. *arXiv preprint arXiv:2302.01318*.

Jian Chen, Yesheng Liang, and Zhijian Liu. 2026a. DFlash: Block diffusion for flash speculative decoding. In *Proceedings of the 43rd International Conference on Machine Learning*, volume 306, pages 15343–15355. PMLR.

Qiaoling Chen, Zijun Liu, Peng Sun, Shenggui Li, Guoteng Wang, Ziming Liu, Yonggang Wen, Siyuan Feng, and Tianwei Zhang. 2026b. ReSpec: Towards optimizing speculative decoding in reinforcement learning systems. In *Proceedings of Machine Learning and Systems*, volume 8.

Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun, Lukasz Kaiser, Matthias Plappert, Jerry Tworek, Jacob Hilton, Reiichiro Nakano, Christopher Hesse, and John Schulman. 2021. Training verifiers to solve math word problems. *arXiv preprint arXiv:2110.14168*.

Wei Fu, Jiaxuan Gao, Xujie Shen, et al. 2025. AReaL: A large-scale asynchronous reinforcement learning system for language reasoning. *arXiv preprint arXiv:2505.24298*.

Daya Guo, Dejian Yang, Haowei Zhang, et al. 2025. DeepSeek-R1: Incentivizing reasoning capability in LLMs via reinforcement learning. *arXiv preprint arXiv:2501.12948*.

Zhenyu He, Zexuan Zhong, Tianle Cai, Jason D. Lee, and Di He. 2024. REST: Retrieval-based speculative decoding. In *Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics (NAACL)*.

Qinghao Hu, Shang Yang, Junxian Guo, Xiaozhe Yao, Yujun Lin, Yuxian Gu, Han Cai, Chuang Gan, Ana Klimovic, and Song Han. 2026. Taming the long-tail: Efficient reasoning RL training with adaptive drafter. In *Proceedings of the 31st ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2*.

Hayate Iso, Tiyasa Mitra, Sudipta Mondal, et al. 2026. Accelerating rl post-training rollouts via system-integrated speculative decoding. *Preprint*, arXiv:2604.26779.

Yoon Kim, Sam Wiseman, Andrew C. Miller, David Sontag, and Alexander M. Rush. 2018. Semi-amortized variational autoencoders. In *Proceedings of the 35th International Conference on Machine Learning (ICML)*.

Kimi Team. 2025. Kimi k1.5: Scaling reinforcement learning with LLMs. *arXiv preprint arXiv:2501.12599*.

Ben Krause, Emmanuel Kahembwe, Iain Murray, and Steve Renals. 2018. Dynamic evaluation of neural sequence models. In *Proceedings of the 35th International Conference on Machine Learning (ICML)*.

Yaniv Leviathan, Matan Kalman, and Yossi Matias. 2023. Fast inference from transformers via speculative decoding. In *Proceedings of the 40th International Conference on Machine Learning (ICML)*.

Yuhui Li, Fangyun Wei, Chao Zhang, and Hongyang Zhang. 2024. EAGLE: Speculative sampling requires rethinking feature uncertainty. In *Proceedings of the 41st International Conference on Machine Learning (ICML)*.

Yuhui Li, Fangyun Wei, Chao Zhang, and Hongyang Zhang. 2025. EAGLE-3: Scaling up inference acceleration of large language models via training-time test. *arXiv preprint arXiv:2503.01840*.

Bingshuai Liu, Ante Wang, Zijun Min, Liang Yao, Haibo Zhang, Yang Liu, Xu Han, Peng Li, Anxiang Zeng, and Jinsong Su. 2026. SPEC-RL: Accelerating on-policy reinforcement learning with speculative rollouts. *Preprint*, arXiv:2509.23232. ArXiv version 3.

Xiaoxuan Liu, Lanxiang Hu, Peter Bailis, Alvin Cheung, Zhijie Deng, Ion Stoica, and Hao Zhang. 2024. Online speculative decoding. In *Proceedings of the 41st International Conference on Machine Learning (ICML)*.

Xupeng Miao, Gabriele Oliaro, Zhihao Zhang, et al. 2024. SpecInfer: Accelerating large language model serving with tree-based speculative inference and

verification. In *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*.

Gabriele Oliaro, Zhihao Jia, Daniel Campos, and Aurick Qiao. 2024. SuffixDecoding: A model-free approach to speeding up large language model inference. *arXiv preprint arXiv:2411.04975*.

Zelei Shao, Vikranth Srivatsa, Sanjana Srivastava, Qingyang Wu, Alpay Ariyak, Xiaoxia Wu, Ameen Patel, Jue Wang, Percy Liang, Tri Dao, Ce Zhang, Yiying Zhang, Ben Athiwaratkun, Chenfeng Xu, and Junxiong Wang. 2025. Beat the long tail: Distribution-aware speculative decoding for RL training. *arXiv preprint arXiv:2511.13841*.

Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Xiao Bi, Haowei Zhang, Mingchuan Zhang, Y. K. Li, Y. Wu, and Daya Guo. 2024. DeepSeekMath: Pushing the limits of mathematical reasoning in open language models. *arXiv preprint arXiv:2402.03300*.

Guangming Sheng, Chi Zhang, Zilingfeng Ye, Xibin Wu, Wang Zhang, Ru Zhang, Yanghua Peng, Haibin Lin, and Chuan Wu. 2025. HybridFlow: A flexible and efficient RLHF framework. In *Proceedings of the Twentieth European Conference on Computer Systems (EuroSys)*.

Yu Sun, Xiaolong Wang, Zhuang Liu, John Miller, Alexei Efros, and Moritz Hardt. 2020. Test-time training with self-supervision for generalization under distribution shifts. In *Proceedings of the 37th International Conference on Machine Learning (ICML)*.

Zili Wang, Zhaopeng Qiu, Yuekai Zhang, Shuang Yu, and Junjie Lai. 2026. Online draft co-training for speculative decoding in large-scale, long-context rl post-training. *arXiv preprint arXiv:2609.07108*.

Qiying Yu, Zheng Zhang, Ruofei Zhu, Yufeng Yuan, Xiaochen Zuo, et al. 2025. DAPO: An open-source llm reinforcement learning system at scale. *arXiv preprint arXiv:2503.14476*.

Weihao Zeng, Yuzhen Huang, Qian Liu, Wei Liu, Keqing He, Zejun Ma, and Junxian He. 2025. Simplerl-zoo: Investigating and taming zero reinforcement learning for open base models in the wild. *arXiv preprint arXiv:2503.18892*.

Yizhou Zhang, Ning Lv, Teng Wang, and Jisheng Dang. 2026. FastGRPO: Accelerating policy optimization via concurrency-aware speculative decoding and online draft learning. In *The Fourteenth International Conference on Learning Representations*.

Yongchao Zhou, Kaifeng Lyu, Ankit Singh Rawat, Aditya Krishna Menon, Afshin Rostamizadeh, Sanjiv Kumar, Jean-François Kagy, and Rishabh Agarwal. 2024. DistillSpec: Improving speculative decoding via knowledge distillation. In *International Conference on Learning Representations (ICLR)*.

# A Implementation Detail

**Training pipeline.** We format each example with the model's native chat template and request step-by-step reasoning followed by a boxed answer. For each prompt $g$, we form a GRPO group of $G = 8$ responses and use $R = R_{\text{acc}} + 0.2R_{\text{fmt}}$, where answer correctness is determined by symbolic equivalence and $R_{\text{fmt}}$ rewards the expected response format. Rewards are standardized independently within each group. Within each drafting setup, the baseline and SPARK use the same target update, sampling configuration, and initialization. Table 3 summarizes the shared settings.

**Draft architectures and training.** We evaluate SPARK with both tree- and block-based drafting. The EAGLE3-style drafter uses one decoder layer, shares the target embedding and language-model head, and proposes a token tree over the full target vocabulary. Its draft objective is $2\mathcal{L}_{\text{SmoothL1}} + 0.1\mathcal{L}_{\text{softCE}}$, combining next-feature prediction and target-distribution distillation. The DFlash-style drafter uses three layers and a block size of 16 (one anchor and 15 predicted positions). It consumes target features, shares the target embedding and language-model head, and is trained with position-weighted soft cross-entropy. At inference, it forms greedy block proposals that are checked by target-model rejection sampling. Both drafters are pretrained on ShareGPT for one epoch and continue to be updated online. For each architecture, the baseline and SPARK start from the same target and pretrained draft checkpoints and use the same draft-training objective.

**SPARK implementation.** For each prompt group $g$, SPARK maintains a rollout-local correction $B_{g,t}$ shared only by the $G$ responses generated from that prompt. After verification round $t$, we collect the eligible contexts in $C_{g,t}$. For tree drafting, these comprise states on the verified path and eligible rejected children; for block drafting, they comprise the accepted prefix and the first rejected token. Each context approximates the forward-KL gradient on the union of the target and draft top-16 tokens, together with a tail bucket. Contributions are aggregated within the prompt group and normalized by its total feedback weight $W_{g,t}$ before updating $B_{g,t}$. Feedback is therefore shared among responses to the same prompt, but never across prompts, and $B_{g,0}$ is reset to zero at the start of each rollout.

Table 3: **Implementation settings.** Settings are shared by a baseline and its SPARK counterpart unless stated otherwise.

| Setting | Value |
|---|---|
| GRPO group size $G$ | 8 responses per prompt |
| Target / online draft learning rate | $1 \times 10^{-6}$ / $1 \times 10^{-4}$ |
| GRPO $\epsilon$ / $\beta$ | 0.1 / 0.04 |
| Sampling | temperature 1.0, top-$p$ 0.95 |
| Maximum prompt / response length | 2,048 / 2,048 tokens |
| Draft pretraining | ShareGPT, 1 epoch |
| Pretraining learning rate / length | $5 \times 10^{-5}$ / 2,048 tokens |
| SPARK rank $r$ / feedback top-$k$ | 8 / 16 |
| Adaptation / projector learning rate | 0.01 / $1 \times 10^{-4}$ |
| Feedback weights (path / rejected) | 1.0 / 1.0 |
| Update / scope / reset | every round / prompt group / rollout |

The projection $A$ is shared across prompt groups and retained across rollouts. We initialize it from evenly spaced rows of the target output projection and orthonormalize it with QR decomposition. Both $A$ and $B_{g,t}$ are maintained in FP32. Gradients for $A$ are accumulated during generation and applied only at the existing draft optimizer step; neither $A$ nor the draft backbone is updated during rollout generation. Target probabilities are reused from verification, so SPARK requires no additional target forward or backward pass.

**Timing protocol.** A baseline and its SPARK counterpart use identical initialization, training, and decoding settings. Generation throughput includes proposal construction, target verification, and SPARK updates. End-to-end time additionally includes online draft training and target-policy optimization; thus, all SPARK runtime overheads are included in the reported results.

# B Theoretical Properties of SPARK

This appendix states and proves the properties of SPARK used in Section 3, with the notation of §3.1. We write $q_c = \mathrm{softmax}(\widetilde{z}_c)$ for the corrected draft distribution, $\bar{\delta}_c = q_c - p_c$ for the full logit gradient, and $\delta_c = \bar{\delta}_c \odot \mathbf{1}_{S_c}$ for the sparse feedback. For a token set $S$, $S^{\mathsf{c}}$ is its complement and $p(S) = \sum_{v \in S} p(v)$. Within a rollout, $A$ is fixed, so $u_c = A^\top h_c$ does not change once $c$ has been drafted.

## B.1 The Feedback Objective

**Lemma B.1** (Logit gradient)**.** *For a fixed distribution $p$ and $q = \mathrm{softmax}(\widetilde{z})$,*

$$\nabla_{\widetilde{z}} D_{\mathrm{KL}}(p \,\|\, q) = q - p.$$

*Proof.* Since $\log q(v) = \widetilde{z}(v) - \log \sum_u e^{\widetilde{z}(u)}$,

$$D_{\mathrm{KL}}(p\|q) = \sum_v p(v) \log p(v) - \langle p, \widetilde{z} \rangle + \log \sum_u e^{\widetilde{z}(u)}.$$

The first term is constant in $\widetilde{z}$, the second has gradient $-p$, and the third has gradient $q$. □

A *miss* occurs when the token selected by the target lies outside the set $K$ of proposed tokens (Section 2), which happens with probability $1 - p(K)$. The next result explains why SPARK uses the forward KL.

**Proposition B.2** (Coverage)**.** *Let $p, q$ be distributions over the vocabulary.*
*(i) For every token $v$,*

$$D_{\mathrm{KL}}(p\|q) \;\geq\; \mathrm{kl}\big(p(v) \,\|\, q(v)\big),$$

*where* $\mathrm{kl}(a\|b) = a \log \frac{a}{b} + (1-a) \log \frac{1-a}{1-b}$. *Hence* $D_{\mathrm{KL}}(p\|q) \to \infty$ *as* $q(v) \to 0$ *whenever* $p(v) > 0$, *whereas* $D_{\mathrm{KL}}(q\|p)$ *can stay finite when* $q(v) = 0$.
*(ii) For every proposal set $K$,*

$$1 - p(K) \;\leq\; 1 - q(K) + \sqrt{\tfrac{1}{2} D_{\mathrm{KL}}(p\|q)}.$$

*Proof.* (i) Merging the vocabulary into the two outcomes $\{v\}$ and $\{v\}^{\mathsf{c}}$ cannot increase the KL divergence (data-processing inequality), which gives the bound. Its first term diverges as $q(v) \to 0$ when $p(v) > 0$. Conversely, if $q(v) = 0$ and $p > 0$ on the support of $q$, then $D_{\mathrm{KL}}(q\|p) = \sum_{u \neq v} q(u) \log \frac{q(u)}{p(u)} < \infty$.
(ii) By Pinsker's inequality,

$$|p(K) - q(K)| \leq \mathrm{TV}(p, q) \leq \sqrt{\tfrac{1}{2} D_{\mathrm{KL}}(p\|q)}. \;\square$$

Part (i) shows that the forward KL strongly penalizes a draft that assigns little probability to a

token the target considers likely, which is the failure that causes a miss; the reverse KL does not. Part (ii) shows that when the proposals carry most of the draft mass ($q(K) \approx 1$), reducing $D_{\mathrm{KL}}(p\|q)$ tightens an upper bound on the miss probability.

### B.2 Gradients and Sparse Feedback

**Lemma B.3** (Parameter gradients)**.** *For a context $c$ of prompt $g$, with $\widetilde{z}_c = z_c + B_{g,t}A^\top h_c$,*

$$\nabla_{B_{g,t}} \mathcal{L}_c = \bar{\delta}_c \, u_c^\top,$$

*and, treating $B_{g,t}$ as fixed,*

$$\nabla_A \mathcal{L}_c = h_c \big(B_{g,t}^\top \bar{\delta}_c\big)^\top.$$

*Eqs. (5) and (6) use these gradients with $\bar{\delta}_c$ replaced by $\delta_c$.*

*Proof.* By Lemma B.1, $d\mathcal{L}_c = \langle \bar{\delta}_c, d\widetilde{z}_c \rangle$. For $B$, $d\widetilde{z}_c = (dB)\, u_c$, so $d\mathcal{L}_c = \langle \bar{\delta}_c u_c^\top, dB \rangle$. For $A$ with $B$ fixed, $d\widetilde{z}_c = B(dA)^\top h_c$, so

$$d\mathcal{L}_c = \langle B^\top \bar{\delta}_c, (dA)^\top h_c \rangle = \langle h_c (B^\top \bar{\delta}_c)^\top, dA \rangle.$$

□

The gradient for $A$ ignores the dependence of $B_{g,t}$ on $A$ through earlier features $u_{c'}$. SPARK uses this truncated gradient, which is why Eq. (6) treats $B_{g,t}$ as fixed.

**Proposition B.4** (Sparse feedback)**.** *(i) For each context $c$, $-\delta_c u_c^\top$ is a descent direction for $\mathcal{L}_c$:*

$$\big\langle \nabla_B \mathcal{L}_c, \; \delta_c u_c^\top \big\rangle = \|u_c\|^2 \sum_{v \in S_c} \bar{\delta}_c(v)^2 \;\geq\; 0.$$

*(ii) Its error is bounded by the mass outside $S_c$:*

$$\big\| (\bar{\delta}_c - \delta_c)\, u_c^\top \big\|_F \leq \|u_c\| \, \epsilon_c,$$

*where $\epsilon_c = p_c(S_c^{\mathsf{c}}) + q_c(S_c^{\mathsf{c}})$, and*

$$p_c(S_c^{\mathsf{c}}) \leq 1 - p_c\big(\mathrm{TopK}(p_c, k)\big),$$
$$q_c(S_c^{\mathsf{c}}) \leq 1 - q_c\big(\mathrm{TopK}(q_c, k)\big).$$

*(iii) Let $\mathcal{L}_{g,t} = \sum_{c \in C_{g,t}} \frac{w_c}{W_{g,t}} \mathcal{L}_c$, with gradient $\nabla$, and let $\widehat{\nabla} = \sum_c \frac{w_c}{W_{g,t}} \delta_c u_c^\top$ be the update direction of Eq. (5). If $\|\widehat{\nabla} - \nabla\|_F < \|\nabla\|_F$, then $-\widehat{\nabla}$ is a descent direction for $\mathcal{L}_{g,t}$.*

*Proof.* (i) By Lemma B.3, the inner product equals $\|u_c\|^2 \langle \bar{\delta}_c, \delta_c \rangle$, and $\langle \bar{\delta}_c, \delta_c \rangle = \sum_{v \in S_c} \bar{\delta}_c(v)^2$.

(ii) The left side equals $\|u_c\| \, \|\bar{\delta}_c \odot \mathbf{1}_{S_c^{\mathsf{c}}}\|_2$, and

$$\|\bar{\delta}_c \odot \mathbf{1}_{S_c^{\mathsf{c}}}\|_2 \leq \sum_{v \notin S_c} |q_c(v) - p_c(v)| \leq \epsilon_c.$$

The tail bounds hold because $S_c$ contains both top-$k$ sets.

(iii) By Cauchy–Schwarz,

$$\begin{aligned} \langle \widehat{\nabla}, \nabla \rangle &= \|\nabla\|_F^2 + \langle \widehat{\nabla} - \nabla, \nabla \rangle \\ &\geq \|\nabla\|_F \big( \|\nabla\|_F - \|\widehat{\nabla} - \nabla\|_F \big) > 0. \end{aligned}$$

□

When $p_c$ is truncated by top-$p$ sampling to at most $k$ tokens, $p_c(S_c^{\mathsf{c}}) = 0$, and the error comes only from the draft tail.

### B.3 What the Correction Computes

Unrolling Eq. (5) from $B_{g,0} = 0$ gives a closed form for the correction. Let

$$\kappa(c, c') = \langle u_c, u_{c'} \rangle = h_c^\top A A^\top h_{c'},$$

and let $\mathcal{C}_{g,t} = \bigcup_{s<t} C_{g,s}$ be the feedback contexts of prompt $g$ before round $t$, with $\lambda_c = \eta \, w_c / W_{g,s}$ for $c \in C_{g,s}$.

**Proposition B.5** (Closed form)**.** *For any context $c'$ of prompt $g$ drafted in round $t$,*

$$B_{g,t} \, u_{c'} = - \sum_{c \in \mathcal{C}_{g,t}} \lambda_c \, \kappa(c, c') \, \delta_c.$$

*Consequently:*
*(i)* Token locality. *The correction changes the logit of a token $v$ only if $v \in S_c$ for some $c \in \mathcal{C}_{g,t}$. The probability ratio of any two tokens outside this set is unchanged.*
*(ii)* Prompt isolation. *$B_{g,t}$ depends only on feedback from responses to prompt $g$. Feedback from one response affects the other responses to the same prompt and no other prompt.*
*(iii)* Context dependence. *Feedback from $c$ is applied at $c'$ with weight $\kappa(c, c')$. If $u_{c'}$ is orthogonal to $u_c$ for all $c \in \mathcal{C}_{g,t}$, the correction at $c'$ is zero.*
*(iv)* Initialization. *With the orthonormal initialization of $A$ (§3.4), $AA^\top$ is the orthogonal projector onto the span of $r$ token-embedding directions. Then $\kappa(c, c')$ is the inner product of the two draft features after projection onto that subspace.*

*Proof.* By induction on $t$, using $B_{g,0} = 0$ and Eq. (5),

$$B_{g,t} = - \sum_{c \in \mathcal{C}_{g,t}} \lambda_c \, \delta_c u_c^\top.$$

Each $u_c$ is unchanged after $c$ is drafted because $A$ is fixed during the rollout. Multiplying by $u_{c'}$ gives the formula. (i) Each $\delta_c$ is supported on $S_c$, so $B_{g,t} u_{c'}$ is supported on their union. If two tokens $v, v'$ receive no correction, then $q_{c'}(v)/q_{c'}(v') =$

$e^{z_{c'}(v)-z_{c'}(v')}$, as without correction. (ii) Only contexts of prompt $g$ appear in $\mathcal{C}_{g,t}$. (iii) Immediate from the formula. (iv) If $A$ has orthonormal columns, $AA^\top$ is the orthogonal projector onto $\mathrm{span}(A)$, which by construction is spanned by the selected rows of $W$. □

Proposition B.5 makes precise how one $B_{g,t}$ acts differently in different contexts (§3.2). SPARK behaves like a kernel smoother over the feedback collected so far in the rollout, with the kernel defined by $A$. The correction is also local by construction: it moves only tokens that verification has already exposed for this prompt, and the reset $B_{g,0} = 0$ discards this memory at the rollout boundary.

**Remark B.6** (Effect of learning $A$)**.** Substituting the closed form of $B_{g,t}$ into Eq. (6) gives

$$G^A_{c'} = -\sum_{c\in\mathcal{C}_{g,t}} \lambda_c \langle \delta_c, \delta_{c'} \rangle \, h_{c'} u_c^\top .$$

A plain gradient step $A \leftarrow A - \alpha G^A_{c'}$ changes the projected feature of $c'$ by

$$\Delta u_{c'} = \alpha \|h_{c'}\|^2 \sum_{c\in\mathcal{C}_{g,t}} \lambda_c \langle \delta_c, \delta_{c'} \rangle \, u_c.$$

To first order, and ignoring the simultaneous change of the earlier $u_c$, this increases $\kappa(c, c')$ when the feedback at $c$ and $c'$ agrees ($\langle \delta_c, \delta_{c'} \rangle > 0$) and decreases it when the feedback conflicts. Across rollouts, $A$ therefore learns which contexts should share corrections. This interprets the update direction; it is not a convergence result, and in practice $A$ is updated by the draft optimizer.

### B.4 Exactness

Let $\mathcal{F}_t$ denote all information available in the current rollout before round $t$. It contains the prompts and the committed tokens, proposals, verifier outputs and target distributions of rounds $0, \ldots, t-1$. It also contains the parameters fixed before the rollout: the target policy, the draft model and $A$.

**Assumption B.7** (Exact verification)**.** In each round, the verifier receives the proposals of every active response together with the distributions they were drawn from. Its randomness is independent of $\mathcal{F}_t$ and of the draft's randomness. Suppose the proposals of round $t$ are determined by $\mathcal{F}_t$ and independent draft randomness. Then, conditionally on $\mathcal{F}_t$ and on the tokens committed before it in that round, each token committed in round $t$ for response $i$ follows the target's sampling distribution given the prefix of response $i$. Tokens committed for different responses are conditionally independent given $\mathcal{F}_t$.

Assumption B.7 is the standard guarantee of lossless speculative decoding. It is also what makes the baseline drafter exact, since that drafter conditions on previously committed tokens as well.

**Theorem B.8** (Exactness)**.** *Under Assumption B.7, the responses generated with SPARK have the same joint distribution as autoregressive sampling from the target. Given the prompts, the responses are independent, and each follows the target's sampling distribution. In particular, the distribution is the same as without SPARK.*

*Proof. Step 1.* $A$ is updated only at the draft optimizer step after the rollout, so it is fixed during the rollout and is part of $\mathcal{F}_0$.

*Step 2.* $B_{g,t}$ is determined by $\mathcal{F}_t$. This holds for $t = 0$, since $B_{g,0} = 0$. By Eq. (5), $B_{g,t+1}$ is a function of $B_{g,t}$ and of $(w_c, \delta_c, u_c)$ for $c \in C_{g,t}$. These quantities are computed from round $t$, so they are determined by $\mathcal{F}_{t+1}$. The claim follows by induction.

*Step 3.* In round $t$, $h_c$ and $z_c$ are computed from the committed prefixes and the current draft tree. They are therefore determined by $\mathcal{F}_t$ and the draft's randomness. By Steps 1 and 2, the same holds for $u_c$, $\widetilde{z}_c$ and $q_c$, and hence for the proposals. SPARK passes $q_c$ to the verifier wherever the acceptance rule uses the proposal distribution.

*Step 4.* By Assumption B.7, each committed token follows, conditionally on everything generated before it, the target's sampling distribution given its own prefix, independently across responses. This conditional law depends on the past only through the token's own prefix. By the chain rule of probability, the joint distribution of all responses is the product of the target's sequence distributions. The same argument applies without SPARK, so the two distributions coincide. □

**Corollary B.9** (GRPO is unaffected)**.** *The rewards, group-normalized advantages and GRPO policy-gradient estimate are functions of the prompts, the sampled responses and the target's log-probabilities. By Theorem B.8, their joint distribution is the same with and without SPARK. SPARK can change the wall-clock time of a GRPO step, but not the distribution of the update it computes.*

**Remark B.10** (Scope)**.** (i) The guarantee is distributional. Individual trajectories differ from the

Table 4: **Sensitivity to correction rank and feedback size.** The default settings are $r = 8$ and $k = 16$. Each sweep varies one parameter while keeping the other fixed.

| Parameter | Value | AAL | E2E (×) |
|---|---|---|---|
| Rank $r$ | 4 | 3.20 | 1.99 |
| | 8 | 3.28 | **2.02** |
| | 16 | 3.29 | 1.97 |
| | 32 | **3.31** | 1.98 |
| | 64 | 3.30 | 1.96 |
| Top-$k$ | 8 | 3.24 | 1.99 |
| | 16 | 3.28 | **2.02** |
| | 32 | **3.29** | 1.97 |
| | 64 | 3.26 | 1.95 |

baseline, because different proposals consume randomness differently. (ii) The proof requires only that the correction used in round $t$ be determined before round $t$ is verified. It therefore also covers delayed or asynchronous updates, in which round $t$ uses $B_{g,s}$ for some $s \leq t$. (iii) The guarantee would not hold if the correction used information produced while verifying the current round, such as the target's own sample, before that round is committed. SPARK does not do this.

### B.5 Cost

**Remark B.11** (Computation and memory)**.** For each context, computing $u_c$ costs $O(dr)$ and applying $B_{g,t}u_c$ costs $O(Vr)$. The draft output head costs $O(Vd)$, so the relative overhead is $O(r/d)$. By Proposition B.5(i), $B_{g,t}$ has nonzero rows only for tokens in the feedback sets $S_c$ of the current rollout. The correction can therefore also be evaluated in $O(dr + m_{g,t}r)$, where $m_{g,t}$ is the number of such tokens and $m_{g,t} \leq \min(V, 2k|\mathcal{C}_{g,t}|)$. Each update in Eq. (5) costs $O(2kr)$ per feedback context and needs no backward pass. Memory is one $V \times r$ matrix per prompt and one $d \times r$ projection.

## C Sensitivity to Correction Rank and Feedback Size

We examine the sensitivity of SPARK to the correction rank $r$ and the number of feedback tokens $k$ using Qwen2.5-3B-Instruct on GSM8K with EAGLE3-style drafting. We vary one parameter at a time, keeping $k = 16$ for the rank sweep and $r = 8$ for the top-$k$ sweep. The default configuration uses $r = 8$ and $k = 16$. Table 4 reports average acceptance length (AAL) and end-to-end (E2E) training speedup over autoregressive GRPO.

**Correction rank.** Increasing the rank from 4 to 32 improves AAL from 3.20 to 3.31, followed by a slight decrease to 3.30 at rank 64. However, the highest E2E speedup is achieved at $r = 8$ (2.02×). Larger ranks provide only small additional gains in acceptance and do not improve overall training speedup.

**Feedback size.** With $k = 8$, SPARK achieves an AAL of 3.24 and an E2E speedup of 1.99×. Increasing $k$ to 16 improves both metrics to 3.28 and 2.02×. The highest AAL (3.29) occurs at $k = 32$, but E2E speedup decreases to 1.97×; increasing $k$ further to 64 reduces both metrics. These results suggest that using more feedback tokens does not necessarily yield better end-to-end efficiency.

Overall, $r = 8$ and $k = 16$ achieve the best E2E speedup among the tested settings while retaining AAL close to the maximum observed in each sweep. We therefore adopt this configuration as the default in our experiments.